

Article

Synthesis of Self-Checking Circuits for Train Route Traffic Control at Intermediate Stations with Control of Calculations Based on Weight-Based Sum Codes

Dmitry V. Efanov ^{1,*} , Artyom V. Pashukov ², Evgenii M. Mikhailiuta ¹, Valery V. Khóroshev ² , Ruslan B. Abdullaev ³, Dmitry G. Plotnikov ¹, Aushra V. Banite ¹, Alexander V. Leksashov ¹, Dmitry N. Khomutov ¹, Dilshod Kh. Baratov ³ and Davron Kh. Ruziev ³

¹ Laboratory “Industrial Stream Data Processing Systems”, Higher School of Mechanical Engineering, Material and Transport Institute, Peter the Great St. Petersburg Polytechnic University, St. Petersburg 195251, Russia; evgeniymixa@gmail.com (E.M.M.); plotnikov_dg@spbstu.ru (D.G.P.); banite_av@spbstu.ru (A.V.B.); leksashov_av@spbstu.ru (A.V.L.); homutov_dn@spbstu.ru (D.N.K.)

² “Automation, Remote Control and Communication on Railway Transport” Department, Russian University of Transport, Moscow 127994, Russia; art_pash@mail.ru (A.V.P.); hvv91@icloud.com (V.V.K.)

³ “Automation and Remote Control” Department, Tashkent State Transport University, Tashkent 100167, Uzbekistan; ruslan_0507@mail.ru (R.B.A.); baratovdx@yandex.com (D.K.B.); ruziyevddd@gmail.com (D.K.R.)

* Correspondence: efanov_dv@spbstu.ru; Tel.: +7-(911)-709-2164

Abstract: When synthesizing systems for railway interlocking, it is recommended to use automated models to implement the logic of railway automation and remote control units. Finite-state machines (FSMs) can be implemented on any hardware component. When using relay technology, the functional safety of electrical interlocking is achieved by using uncontrolled (safety) relays with a high coefficient of asymmetry of failures in types $1 \rightarrow 0$ and $0 \rightarrow 1$. When using programmable components, the use of backup and diverse protection methods is required. This paper presents a flexible approach to synthesizing FSMs for railway automation and remote control units that offer both individual and route-based control. Unlike existing solutions, this proposal considers the pre-failure states of railway automation and remote control units during the finite-state machine synthesis stage. This enables the implementation of self-checking and self-diagnostic modules to manage automation units. By increasing the number of states for individual devices and considering the states of interconnected objects, the transition graphs can be expanded. This expansion allows for the synthesis of the transition graph of the control subsystem and other systems. The authors used a field-programmable gate array (FPGA) to implement a finite-state machine. In this case, the proposal is to encode the states of a finite-state machine using weight-based sum codes in the residue class ring based on a given modulus. The best coverage of errors occurring at the outputs of the logic converter in the structure of the FSM can be ensured by selecting the weighting coefficients and the value of the module. This paper presents an example of synthesizing an FPGA-based FSM using state encoding through modular weight-based sum codes. The operation of the synthesized device was modeled. It was found to operate according to the same algorithm as the real devices. When synthesizing self-checking and self-controlled train control devices, it is recommended to consider the solutions proposed in this paper.

Keywords: traffic control at railway stations; railway automation and remote control; electrical interlocking; finite-state machine model; self-checking device synthesis; field-programmable gate array



Citation: Efanov, D.V.; Pashukov, A.V.; Mikhailiuta, E.M.; Khóroshev, V.V.; Abdullaev, R.B.; Plotnikov, D.G.; Banite, A.V.; Leksashov, A.V.; Khomutov, D.N.; Baratov, D.K.; et al. Synthesis of Self-Checking Circuits for Train Route Traffic Control at Intermediate Stations with Control of Calculations Based on Weight-Based Sum Codes. *Computation* **2024**, *12*, 171. <https://doi.org/10.3390/computation12090171>

Academic Editor: Filippo Palombi

Received: 1 July 2024

Revised: 31 July 2024

Accepted: 23 August 2024

Published: 26 August 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Railway automation equipment and systems are used to safety control train traffic at stations [1]. Such systems can be implemented on different components. For instance,

the majority of railway stations in Russia have electrical interlocking systems for switches and signals that rely on electromagnetic relays [2]. There are technical solutions in which a part of the circuits, known as the “type-setting group”, responsible for inputting control commands, is based on microelectronic and microprocessor technology. Another part, called the “executive group”, which implements technological control algorithms, is based on relays. Fully microelectronic and microprocessor systems are also available [3]. For instance, the characteristics of interlocking equipment for controlling railway crossings via field-programmable gate arrays (FPGAs) are detailed in [4]. Reference [5] discusses the features of synthesizing track receivers for track circuits, which are sensors used to determine the location of mobile units, on FPGAs. Reference [6] examines a technique for generating self-checking control units for railway switches on FPGAs. These units are equipped with information that connects with monitoring systems designed to assess their technical condition, similar to the system presented in [7]. Additional devices, such as programmable logic controllers and microprocessors [8], are utilized for railway automation’s circuit solutions.

Criteria describing the logic of functioning of interdependent railway automation devices are used when synthesizing control circuits for automation units at stations. For instance, in contemporary electrical interlocking systems, a route is established by pressing the start and end buttons, which flip-flops the railway switches along the route to move automatically. Once all traffic safety criteria are met, a permissive indication is illuminated at the railway signal that encloses the aforementioned route. Thus, when controlling automation units at the station, logic chains of states are used. At the same time, the devices and systems of railway automation and remote control implement finite-state machines (FSMs).

Reference [9] states that relay systems should be deterministic automata. With this in mind, a methodology has been developed for testing railway automation and remote control relay systems using an integrated model of distributed systems, temporal formalism, and the Dedan tool. Relay train control systems are typically asynchronous FSMs. This fact limits their operations. For example, there may be delays in turning relays on or off, or issues with data encoding. These limitations have been noted in previous research [10].

FSM models are frequently used to test systems that implement logical dependencies. For instance, in [11], an electrical interlocking verification is described by means of the FSM model. In this case, the verifier analyzes the contents of the control table for any conflicting settings.

Reference [12] is relevant for promising projects in the field of synthesizing train control systems. It highlights the correlation between living organisms and the operation of FSMs. The authors developed a DNA FSM. A programmable allosteric DNAzyme strategy was developed to build an FSM that can dynamically respond to successive stimulus signals. The results of the work are interesting in terms of the synthesis of FSMs describing the logic behind train traffic control.

In general, any station can be described by a set of standard templates corresponding to the track development. It is not a challenging task to delineate the logical criteria for allocating and executing the routes, mirroring the approach taken in [13]. This approach employs mathematical logic for station-based train traffic control groups. It should be acknowledged that guaranteeing the dependability and safety of railway automation systems through established methods [14,15] is imperative when synthesizing them.

In such works, authors do not consider synthesizing an FSM with self-checking and self-control properties. The use of automata theory is limited to verifying the logic of functioning and control algorithms. This is due to its peculiar application. They do not perform functions that require error detection. This paper presents a methodology for synthesizing self-checking devices for railway automation and remote control. The methodology is based on the FSM model that allocates protective and pre-failure states.

2. Research Objective

Railway automation devices and electrical switch and signal interlocking systems can be represented by a Mealy machine [16,17]:

$$A = \langle X, S, Z, S_1, \varphi, \chi \rangle$$

where $X \in \{X_1, X_2, \dots, X_r\}$ is the set of input variables;

$Q \in \{\Phi, \Psi, \Omega, \Delta\}$ is a set of the machine states, ($\Phi \in \{\Phi_1, \Phi_2, \dots, \Phi_n\}$ is a set of efficient and serviceable states, $\Psi \in \{\Psi_1, \Psi_2, \dots, \Psi_m\}$ is a set of pre-failure serviceable states, $\Omega \in \{\Omega_1, \Omega_2, \dots, \Omega_k\}$ is a set of inoperable protective states, and $\Delta \in \{\Delta_1, \Delta_2, \dots, \Delta_l\}$ is a set of inoperable dangerous states causing a threat of an accident or catastrophe);

$Z \in \{Z_1, Z_2, \dots, Z_p\}$ is a set of output states;

$Q_1 \in Q$ is the initial state of the machine;

φ is a transition function that maps the set $X \times Q$ to the set Q : $X \times Q \rightarrow Q$;

χ is a transition function that maps the set $X \times Q$ to the set Z : $X \times Q \rightarrow Z$.

A methodology for synthesizing self-checking circuits for FPGA-based railway automation and remote control is required. It should consider possible transitions to pre-failure states.

3. General Research Approaches and Methodology

The target of this study is railway automation and remote control systems employed at intermediate railway stations. The scope of this study is the synthesis of control algorithms, with particular consideration given to the interdependencies between switches and signals. Concurrently, the control algorithms consider not only the typical operation of the devices but also their transitions to the so-called pre-failure and protective states.

This study revealed that the FSM model is an effective means of describing the operational logic of railway automation and remote control devices at intermediate railway stations. The complexity lies in the formation of the transition graph of the FSM, which must take into account the states of all devices. It is therefore most efficient to synthesize individual graphs by station route. It should be noted that the number of routes at intermediate stations is relatively limited. It can be reasonably deduced that the transition graph may not contain a substantial number of states.

A standard methodology is employed to synthesize an FSM. However, in order to safeguard the device from faults and errors resulting from stuck-at failures and malfunctions, it is determined that widely underutilized weight-based sum codes in the residue class ring using a specified modulus should be applied. The selection of a weight-based sum code with a specific modulus value and particular error detection characteristics for the information characters of the codewords is dependent upon the number of states, memory cells, and outputs of the device in question. This methodology enables the synthesis of a self-controlled FSM with the capacity to indicate transitions, which is crucial for the operational tasks of the final device. Indeed, it permits the digital simulation of its operation in parallel with the completion of the primary tasks. Such a solution is conducive to subsequent integration with the control systems of the upper hierarchy, specifically the centralized traffic control system.

This paper sets forth the fundamental principles of device synthesis employing programmable components, specifically field-programmable gate arrays (FPGAs). The example of a simple, standard railway station demonstrates how to synthesize a device using a table of routes, interdependencies of switches and signals, and a set of states of railway trackside devices.

This paper does not address the issues of reliability and safety associated with the implementation of upper-level railway automation and remote control systems. These issues are discussed in detail elsewhere. The attainment of a high degree of reliability and safety is contingent upon the utilization of redundancy, diverse protection, and technical diagnostic methodologies.

4. Requirements for the Operating Logic of Railway Trackside Equipment

When implementing a train control system, it is essential to ensure the safety control of switches and signals, which are interdependent. The trackside automation and remote control mechanical units serve as the final control equipment. Any given circuit for their operation can be pre-modelled as an FSM. It is required to describe the finite-state machine logic and move from a verbal description to the synthesis of an FPGA-based self-checking device.

There are interdependencies between trackside units of railway automation and remote control. Both individual and group (route) control devices operate at stations. The concept of a “route” is fundamental in the construction of electrical interlocking.

A route is a structured method for a rolling stock’s following by train or shunting order at a station. There are two types of routes: train routes and shunting routes.

A train route is a route intended for the movement of a rolling stock along the stretches and through stations along prepared routes for receiving, departing, or passing trains.

A shunting route is a designated track for locomotives, with or without cars, to move within the station territory and conduct shunting operations, including departure for the stretch if necessary.

When setting a route, the navigation and security switches are locked to exclude conflicting (secant) routes. Once the route is locked, a permissive indication is activated at the signal that locks the route.

To set a train route in the electrical interlocking, it is necessary to specify the departure and destination points. In certain systems, it may be necessary to manually adjust the switches to the desired positions. In most cases, the route is automatically set by inputting the departure and destination points and intermediate points for longitudinal station on the station’s mnemonic diagram into the automated workstation (AWS) control technology window. To set a shunting route in an electrical interlocking, it is required to specify all intermediate points, typically certain shunting signals.

To comprehend the process of implementing a route, let us consider the most basic intermediate station. Figure 1 displays the plan for a single-line station. There are more than a thousand of such stations across post-Soviet territories. They serve as technologically equipped facilities for crossing and overtaking operations.

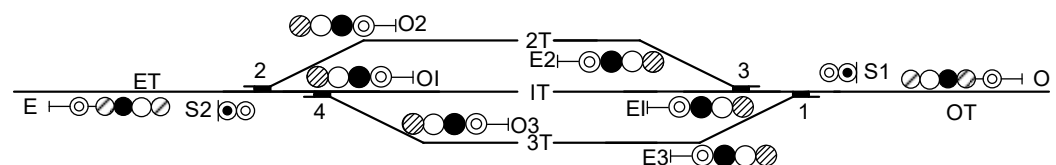


Figure 1. Single-line passing track plan.

The aforementioned station comprises four switches and six output, two input, and two shunting signals. Tables 1 and 2 present a comprehensive overview of all train and shunting routes connected to this station. The following tables display the interdependencies between routes, switches, and signals for train and shunting routes. Table 3 shows the interdependencies of signal indications.

The following designations are accepted in Table 3: ● is the red signal indication; ○ is the green signal indication; ⊗ is the “Two yellow, upper flashing” signal indication; ⊙ is the “Two yellow” signal indication.

Table 1. Table of train routes.

Route	Route Number	Route Name	Traffic Light	Railway Switch Points			
				2	4	1	3
Even yard neck section	1	Reception to T1	E	+	+		
	2	Reception to T2	E	—			
	3	Reception to T3	E	+	—		
	4	Dispatch from T1	O2	+	+		
	5	Dispatch from T2	O1	—			
	6	Dispatch from T3	O3	+	—		
Odd yard neck section	7	Reception to T1	O			+	+
	8	Reception to T2	O			+	—
	9	Reception to T3	O			—	
	10	Dispatch from T1	E1			+	+
	11	Dispatch from T2	E2			+	—
	12	Dispatch from T3	E3			—	

Notes. 1. The symbol “+” indicates the normal (“positive”) position of the arrow; 2. The symbol “—” indicates the translated (“minus”) position of the arrow.

Table 2. Table of shunting routes.

Route		Route Number		Route Name	Route-Defining Railway Switch Point Electric Mechanism
Even yard neck section	From signal	S2	13	To T1	+2 +4
		S2	14	To T2	—2
		S2	15	To T3	+2 —4
		O1	16	Beyond the signal S2	+2 +4
		O3	17	Beyond the signal S2	—2
		O2	18	Beyond the signal S2	+2 —4
Odd yard neck section	From signal	S1	19	To T1	+1 +3
		S1	20	To T2	+1 —3
		S1	21	To T3	—1
		E1	22	Beyond the signal S1	+1 +3
		E2	23	Beyond the signal S1	+1 —3
		E3	24	Beyond the signal S1	—1

To devise a control system for the switches and signals at the intermediate station, it is imperative to synthesize control circuits for individual railway automation units independently and then make their information interface with each other.

When implementing route control, it is important to consider all processes that take place during traffic control at the station. Two notable procedures include the route locking and its subsequent release. In this scenario, the release may occur as a result of either train passage or artificial means (no train passage). This happens when the station attendant cancels the route. There are separate descriptions of these processes during synthesis.

The following parameters are utilized to set the route:

- Route type (train or shunting);
- Departure point (the signal from which the route starts);
- Destination point (the signal to which or beyond which the route is constructed).

Where practicable, stations with alternative routes will display a variation option. There is no such route for the station considered in Figure 1.

There are only 12 train routes and 12 shunting routes for the station shown in Figure 1. Each route can be defined as a system state in which control signals are transmitted to trackside automation equipment and remote control mechanical devices.

Table 3. Table of interdependencies of signal indications.

Route Name	Movement Direction Is Up			
	Railway Signal Indications			
	E	E1	E2	E3
Reception to T1				
Reception to T2	 			
Reception to T3	 			
Pass through T1		 ; 		
Pass through T2	 		 ; 	
Pass through T3	 			 ; 
Dispatch from T1		 ; 		
Dispatch from T2		 ; 		
Dispatch from T3				 ; 
Route Name	Movement Direction is Down			
	Railway Signal Indications			
	O	O1	O2	O3
Reception to T1				
Reception to T2	 			
Reception to T3	 			
Pass through T1		 ; 		
Pass through T2	 		 ; 	
Pass through T3	 			 ; 
Dispatch from T1		 ; 		
Dispatch from T2			 ; 	
Dispatch from T3				 ; 

As previously stated, the route can be released manually or during the train passage. When artificially releasing the route, it is necessary to consider the route type and the condition of the track section in front of the railway signal from which the route is being constructed. This step is required to postpone the route cancellation.

During the passage of the train, the route is released according to the following rules [18]:

- The first section in the train route is released when the approach section is released, this section is occupied, the next section is occupied, and this section is released and remains free for at least 6 s;
- The first section in shunting routes is released when this section is occupied, the next one is occupied, and this section is released and remains free for at least 6 s;
- Any section that is not the first one is released when this section is occupied, the next one is occupied, and this section is released and remains free for at least 6 s;
- If the approach section was not released during the train route, then the route will be released after the train arrives at the route destination point.

The delay in releasing is essential to prevent a temporary loss of shunt sensitivity.

These conditions must be taken into account when synthesizing the railway automation and remote control schemes.

5. Principles of Forming the Transition Graph for an FSM

The input data required for the even bottleneck route locking system are as follows:

- x_1 is the route type (0—not specified; 1—train; 2—shunting);
- x_2 is the route departure point (0—not specified; 1—E; 2—O1; 3—O2; 4—O3; 5—S2);
- x_3 is the route destination point (0—not specified; 1—O; 2—O1; 3—O2; 4—O3; 5—S2);
- x_4 is the track and switch point section occupancy along the route (0—occupied; 1—free);
- x_5 is switch point position 2 (0—plus position; 1—minus position; 2—no control);
- x_6 is switch point position 4 (0—plus position, 1—minus position, 2—no control);
- x_7 is the track and switch point sections locking (0—released; 1—locked);
- x_8 is switch point locking 2 (0—released; 1—locked);
- x_9 is switch point locking 4 (0—released; 1—locked);

The following are the output data required for the system:

- z_1 is switch point state 2;
- z_2 is switch point state 4;
- z_3 is railway signal state E (0—forbidding indication; 1—permissive indication; 2—no indication);
- z_4 is railway signal state O1 (0—forbidding indication; 1—permissive indication; 2—no indication);
- z_5 is railway signal state O2 (0—forbidding indication; 1—permissive indication; 2—no indication);
- z_6 is railway signal state O3 (0—forbidding indication; 1—permissive indication; 2—no indication);
- z_7 is railway signal state S2 (0—forbidding indication; 1—permissive indication; 2—no indication);

Table 4 shows the system states when the route is locked in an even bottleneck.

There are non-binary numbers in the “Parameter Vector” column. In this case, similar parameters, such as signal numbers, are combined. Signal numbers are described by separate variables when implemented on the FPGA. For example, if the departure point of the route is the railway signal O, then the variable responsible for this signal will be assigned “1”, and the rest will be assigned “0”. The same applies to the route destination point. Figure 2 shows the transition graph for a single route.

It should be mentioned that the complete system will comprise multiple transition graphs functioning together. This is because the route-setting process for all possible travel options is identical, involving the selection of the departure and destination points, route preparation, and route locking. For instance, Figure 3 displays the transition graph for two routes.

As can be observed in Figure 3, states Q3 and Q15, which correspond to the second route, have been incorporated. Thus, the addition of new states enables the construction of a transition graph for the entire bottleneck or station.

This paper presents a technique for synthesizing the transition graph of an FSM for a train traffic control system, taking into account route dependencies.

The Synthesizing Process for an FSM Transition Graph Involves the Following Stages:

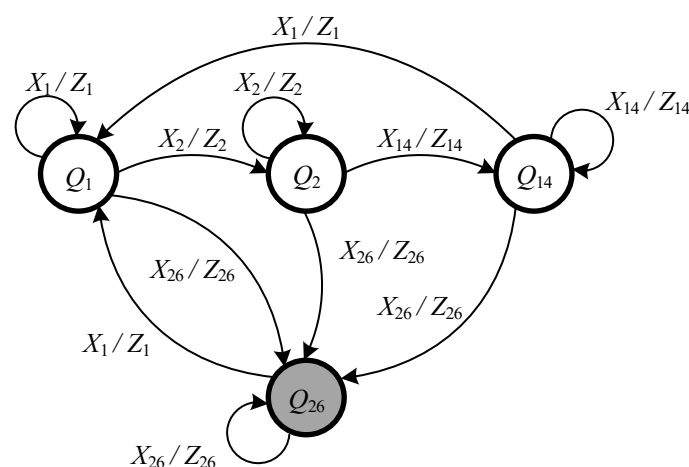
1. For an intermediate station, a switch points and signals interdependencies table is acquired from technical documentation [19].
2. For each individual route, a graph is created that includes the following states:
 - Initial state (in this case, there are no routes at the station);
 - Route preparation state (one state for each route);
 - Route locking state (one state for each route);

- Protective state;
- A state of the system in which one or more of the control devices go into a pre-failure state.

Table 4. Table of states when locking a route.

State	Input Parameter Vectors $\langle x_1 x_2 \dots x_9 \rangle$	Notation for the Parameter Vectors Set	Output Vector $\langle z_1 z_2 \dots z_7 \rangle$	Notation for the Output Vectors Set
Q_1 —The route is released	0 0 0 ~ ~ ~ 0 0 0	X_1	~ ~ 0	Z_1
Route track preparation				
Q_2 —Reception from E to T1	1 1 2 1 0 0 0 0 0	X_2	1 1 0 0 0 0 0	Z_2
Q_3 —Reception from E to T2	1 1 3 1 1 ~ 0 0 0	X_3	2 ~ 0 0 0 0 0	Z_3
Q_4 —Reception from E to T3	1 1 4 1 0 1 0 0 0	X_4	1 2 0 0 0 0 0	Z_4
Q_5 —Dispatch from T1 beyond E	1 2 1 1 0 0 0 0 0	X_5	1 1 0 0 0 0 0	Z_5
Q_6 —Dispatch from T2 beyond E	1 3 1 1 1 ~ 0 0 0	X_6	2 ~ 0 0 0 0 0	Z_6
Q_7 —Dispatch from T3 beyond E	1 4 1 1 0 0 0 0 0	X_7	1 2 0 0 0 0 0	Z_7
Q_8 —From S2 to T1	2 5 2 1 0 0 0 0 0	X_8	1 1 0 0 0 0 0	Z_8
Q_9 —From S2 to T2	2 5 3 1 1 ~ 0 0 0	X_9	2 ~ 0 0 0 0 0	Z_9
Q_{10} —From S2 to T3	2 5 4 1 0 1 0 0 0	X_{10}	1 2 0 0 0 0 0	Z_{10}
Q_{11} —From T1 beyond signal S2	2 2 5 1 0 0 0 0 0	X_{11}	1 1 0 0 0 0 0	Z_{11}
Q_{12} —From T2 beyond signal S2	2 3 5 1 1 ~ 0 0 0	X_{12}	2 ~ 0 0 0 0 0	Z_{12}
Q_{13} —From T3 beyond signal S2	2 4 5 1 0 1 0 0 0	X_{13}	1 2 0 0 0 0 0	Z_{13}
Route locking				
Q_{14} —Reception from E to T1	1 1 2 1 0 0 1 1 1	X_{14}	5 5 1 0 0 0 0	Z_{14}
Q_{15} —Reception from E to T2	1 1 3 1 1 ~ 1 1 1	X_{15}	6 ~ 1 0 0 0 0	Z_{15}
Q_{16} —Reception from E to T3	1 1 4 1 0 1 1 1 1	X_{16}	5 6 1 0 0 0 0	Z_{16}
Q_{17} —Dispatch from T1 beyond E	1 2 1 1 0 0 1 1 1	X_{17}	5 5 0 1 0 0 0	Z_{17}
Q_{18} —Dispatch from T2 beyond E	1 3 1 1 1 ~ 1 1 1	X_{18}	6 ~ 0 0 1 0 0	Z_{18}
Q_{19} —Dispatch from T3 beyond E	1 4 1 1 0 0 1 1 1	X_{19}	5 6 0 0 0 1 0	Z_{19}
Q_{20} —From S2 to T1	2 5 2 1 0 0 1 1 1	X_{20}	5 5 0 0 0 0 1	Z_{20}
Q_{21} —From S2 to T2	2 5 3 1 1 ~ 1 1 1	X_{21}	6 ~ 0 0 0 0 1	Z_{21}
Q_{22} —From S2 to T3	2 5 4 1 0 1 1 1 1	X_{22}	5 6 0 0 0 0 1	Z_{22}
Q_{23} —From T1 beyond signal S2	2 2 5 1 0 0 1 1 1	X_{23}	5 5 0 1 0 0 0	Z_{23}
Q_{24} —From T2 beyond signal S2	2 3 5 1 1 ~ 1 1 1	X_{24}	6 ~ 0 0 1 0 0	Z_{24}
Q_{25} —From T3 beyond signal S2	2 4 5 1 0 1 1 1 1	X_{25}	5 6 0 0 0 1 0	Z_{25}
Q_{26} —Protective state	Ω_1^*	X_{26}	~ ~ 0 0 0 0 0	Z_{26}

Note: The symbol Ω^* represents a set of vectors, $\Omega^* = \Omega \setminus \Omega^+$, where Ω denotes the entire set of code vectors and Ω^+ indicates the state code vectors used in the finite-state machine.

**Figure 2.** Transition graph for a single route.

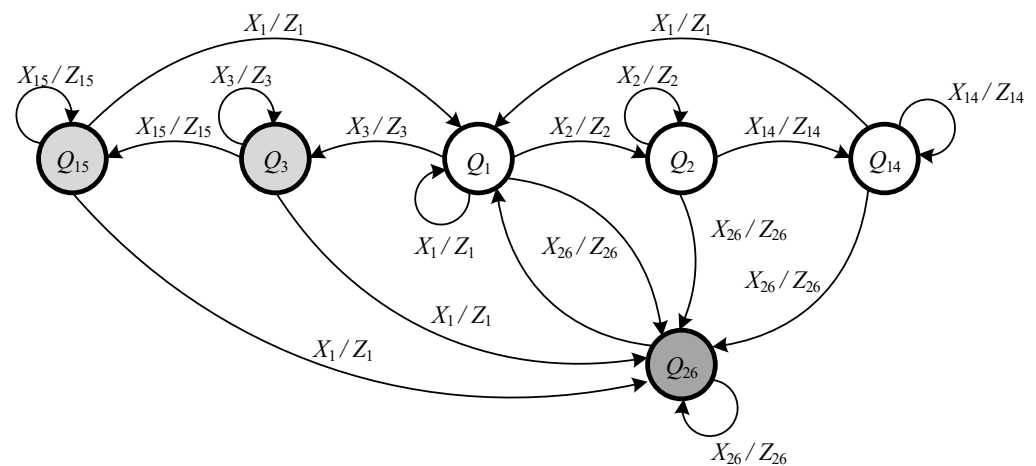


Figure 3. Transition graph for two routes.

At this stage, it is appropriate to specify the initial and protective states for a single route. For the rest, these states can remain unspecified, as they will be combined further.

Cascade connection of all graphs is performed. All states with identical names, whether initial or protective, are combined into a single state, which serves as the origin or destination for state transitions.

If there are multiple routes available, the graph will be supplemented with transitions that connect the states corresponding to a locked route and a new non-conflicting route. If multiple routes cannot be set at a station simultaneously, this step is omitted.

If monitoring is required, additional states can be incorporated to denote the presence of a particular malfunction. This can be applied to both system malfunctions and malfunctions in trackside equipment, using the concept proposed in [20,21]. This will enable the implementation of intelligent control functions in instances of protective and hazardous failures in railway automation equipment.

An example below illustrates FPGA-based device synthesis.

6. FPGA-Based FSM Synthesis

In order to prevent the transfer of switches that are not included in the route, they also need to be controlled. This means that they must hold the position they are in after releasing the route.

Table 4 was updated to include new states for routes that do not have switches included in them. The updated states are shown in Table 5. All vectors are also represented in binary form.

Table 5. State table for setting two routes.

State	Input Parameter Vectors	Notation for the Parameter Vectors Set	Output Vector Values	Notation for the Parameter Vectors Set
Q ₁ —The route is released	00 000 000 ~ ~ ~ 0 0 0	X ₁	001 001 0 0 0 0 0	Z ₁
Route track preparation				
Q ₂ —Reception from E to T1	01 001 010 1 00 00 0 0 0	X ₂	001 001 0 0 0 0 0	Z ₂
Q ₄ —Reception from E to T2	01 001 100 1 00 00 0 0 0	X ₄	001 010 0 0 0 0 0	Z ₃
Route locking				
Q ₁₄ —Reception from E to T1	01 001 010 1 00 00 1 1 1	X ₁₄	101 101 1 0 0 0 0	Z ₁₄
Q ₁₆ —Reception from E to T2	01 001 100 1 01 00 1 1 1	X ₁₆	101 110 1 0 0 0 0	Z ₁₅
Q ₂₆ —Protective state	Ω _i [*]	X ₂₆	000 000 0 0 0 0 0	Z ₂₆

Note: The symbol Ω^{*} represents a set of vectors, Ω^{*} = Ω \ Ω⁺, where Ω denotes the entire set of code vectors and Ω⁺ indicates the state code vectors used in the finite-state machine.

The synthesis process will be carried out in a graphical editor on D flip-flops [22].
Let us compile and minimize the table of FSM transitions and outputs (see Table 6).

Table 6. Minimized table of FSM transitions and outputs.

Q	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}$												A*
	0000	0000	0000	0000	0000	0000	0000	0000	1101	1110	1101	1110	
	000	001	010	011	100	101	110	111	101	100	100	101	
	000	000	000	000	000	000	000	000	000	111	111	111	
1 (Q ₁)	(1), Z ₁	(1), Z ₁	(1), Z ₁	(1), Z ₁	(1), Z ₁	(1), Z ₁	(1), Z ₁	(1), Z ₁	2, Z ₂	3, Z ₄	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆
2 (Q ₂)	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	(2), Z ₂	6, Z ₂₆	4, Z ₁₄	6, Z ₂₆	6, Z ₂₆
3 (Q ₄)	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	(3), Z ₄	6, Z ₂₆	5, Z ₁₆	6, Z ₂₆
4 (Q ₁₄)	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	6, Z ₂₆	6, Z ₂₆	(4), Z ₁₄	6, Z ₂₆	6, Z ₂₆
5 (Q ₁₆)	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	6, Z ₂₆	6, Z ₂₆	6, Z ₂₆	(5), Z ₁₆	6, Z ₂₆
6 (Q ₂₆)	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	1, Z ₁	(6), Z ₂₆	(6), Z ₂₆	(6), Z ₂₆	(6), Z ₂₆	(6), Z ₂₆

The term A* is used to designate all vectors that are not utilized for transition from a specific state in the control system, as it cannot skip any state.

The table is then encoded. Each state is mapped to a binary set. The set “000” corresponds to the protective state. Table 7 shows the codeword formation rules, and Table 8 is the encoded transition table.

Table 7. State encoding.

Q	Encoded State
1	001
2	010
3	011
4	100
5	101
6	000

Table 8. Encoded transition table of an FSM.

Q	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}$												A*
	0000	0000	0000	0000	0000	0000	0000	0000	1101	1110	1101	1110	
	000	001	010	011	100	101	110	111	101	100	100	101	
	000	000	000	000	000	000	000	000	000	111	111	111	
001	(001)	(001)	(001)	(001)	(001)	(001)	(001)	(001)	010	011	000	000	000
010	000	000	000	000	000	000	000	000	(010)	000	100	000	000
011	000	000	000	000	000	000	000	000	000	(011)	000	101	000
100	001	001	001	001	001	001	001	001	000	000	(100)	000	000
101	001	001	001	001	001	001	001	001	000	000	000	(101)	000
000	001	001	001	001	001	001	001	001	(000)	(000)	(000)	(000)	(000)

Once the table of transitions and outputs has been encoded, the functions for activating the memory cells are defined. Table 9 provides the rules derived from the D-flip-flop operation logic when transitioning between states at time points $t - 1$ (previous clock cycle) and t (current clock cycle), based on logical principles.

Table 9. Table of definition of the Y_D functions’ values.

$y(t - 1)$	$y(t)$	
	0	1
0	0	0
1	1	1

The flip-flop activation functions (Table 10) are obtained using the data from Table 9.

Table 10. The Y_D -flip-flop truth table inclusion functions.

Decimal Number of a Binary Vector	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}$	$y_1y_2y_3$	Y_{D1}	Y_{D2}	Y_{D3}	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9	z_{10}	z_{11}
0	0000000000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	0000000000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
2	0000000000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0000000000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0000000000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
5	0000000000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
64	0000001000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
65	0000001000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
66	0000001000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
67	0000001000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
68	0000001000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
69	0000001000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
128	0000010000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
129	0000010000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
130	0000010000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
131	0000010000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
132	0000010000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
133	0000010000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
192	0000011000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
193	0000011000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
194	0000011000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
195	0000011000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
196	0000011000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
197	0000011000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
256	0000100000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
257	0000100000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
258	0000100000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
259	0000100000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
260	0000100000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
261	0000100000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
320	0000101000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
321	0000101000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
322	0000101000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
323	0000101000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
324	0000101000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
325	0000101000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
384	0000110000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
385	0000110000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
386	0000110000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
387	0000110000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
388	0000110000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
389	0000110000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
448	0000111000 000		0	0	1	0	0	0	0	0	0	0	0	0	0	0
449	0000111000 001		0	0	1	0	0	0	0	0	0	0	0	0	0	0
450	0000111000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
451	0000111000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
452	0000111000 100		0	0	1	0	0	0	0	0	0	0	0	0	0	0
453	0000111000 101		0	0	1	0	0	0	0	0	0	0	0	0	0	0
6968	1101100111 000		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6969	1101100111 001		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6970	1101100111 010		1	0	0	1	0	1	1	0	1	1	0	0	0	0
6971	1101100111 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6972	1101100111 100		1	0	0	1	0	1	1	0	1	1	0	0	0	0

Table 10. Cont.

Decimal Number of a Binary Vector	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}$	$y_1y_2y_3$	Y_{D1}	Y_{D2}	Y_{D3}	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9	z_{10}	z_{11}
6973	1101100111 101		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6976	1101101000 000		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6977	1101101000 001		0	1	0	0	0	1	0	0	1	0	0	0	0	0
6978	1101101000 010		0	1	0	0	0	1	0	0	1	0	0	0	0	0
6979	1101101000 011		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6980	1101101000 100		0	0	0	0	0	0	0	0	0	0	0	0	0	0
6981	1101101000 101		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7488	1110101000 000		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7489	1110101000 001		0	1	1	0	0	1	0	1	0	0	0	0	0	0
7490	1110101000 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7491	1110101000 011		0	1	1	0	0	1	0	1	0	0	0	0	0	0
7492	1110101000 100		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7493	1110101000 101		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7544	1110101111 000		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7545	1110101111 001		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7546	1110101111 010		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7547	1110101111 011		1	0	1	1	0	1	1	1	0	1	0	0	0	0
7548	1110101111 100		0	0	0	0	0	0	0	0	0	0	0	0	0	0
7549	1110101111 101		1	0	1	1	0	1	1	1	0	1	0	0	0	0
-	A^{**}		0	0	0	0	0	0	0	0	0	0	0	0	0	0

All code vectors corresponding to the intersections of A^* and y_i are denoted by A^{**} .
Write out the input sets where the functions $Y_{Di} = 1$:

$$Y_{D1} = \{6970, 6972, 7547, 7549\}$$

$$Y_{D2} = \{6977, 6978, 7489, 7491\}$$

$$Y_{D3} = \{0, 1, 4, 5, 64, 65, 68, 69, 128, 129, 132, 133, 192, 193, 196, 197, 256, 257, 260, 261, 320, 321, 324, 325, 384, 385, 388, 389, 448, 449, 452, 453, 7489, 7491, 7547\}$$

Figure 4 shows a block diagram of a finite-state machine that controls trackside equipment in order to set two routes. There are no outputs Z_8 – Z_{11} in the figure since these entities do not contribute to determining the routes being examined, and their functionalities are negligible.

The operation of the FSM during the route setting, locking, and return to the initial state is demonstrated in Figure 5 (up to the red line). As can be observed, the system initially arranges the route by positioning the switches. Then, it locks certain route sections, followed by the necessary signal activation. The figure also illustrates the process of transitioning the system to a protective state (from the red line) and returning it to its original state (from the green line) when input data that do not align with the system's logic are received. It is also noteworthy that the transition of the system to a protective state is also possible when setting a route. A route's switch losing control serves as an example.

Table 11 shows the parameters of the control system designed using the Cyclone V FPGA [23]. In the FPGA, only three blocks are occupied. It is evident that separate logic cells are occupied in each block (logic utilization (in ALMs)) when viewed in detail. The number of cells occupied in each block may vary. In this case, the number varies from 0 to 10. The same applies to input/output pins. They are shown in brown text in the figure.

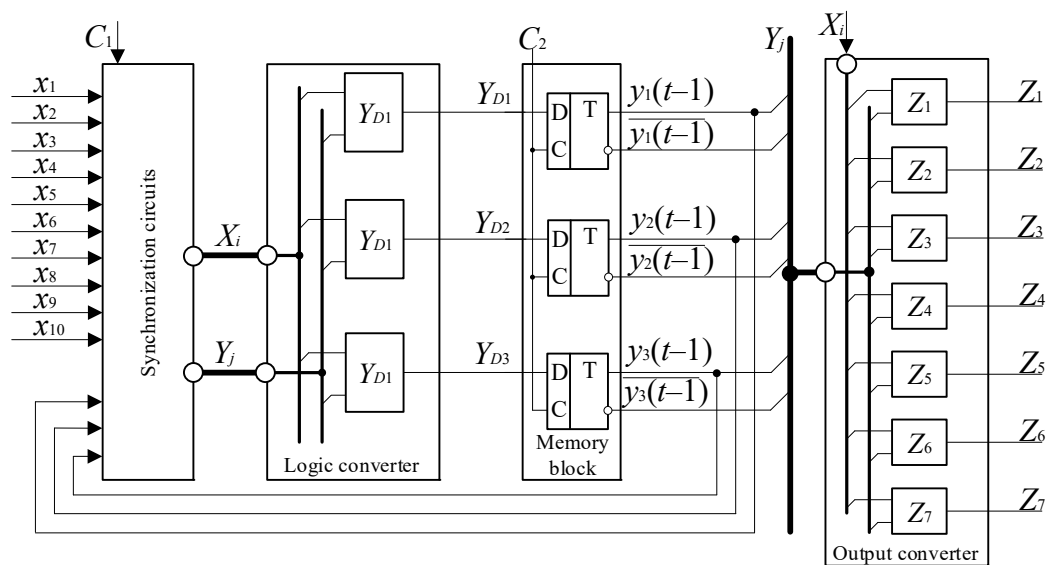


Figure 4. Block diagram of a D-flip-flop-based FSM.

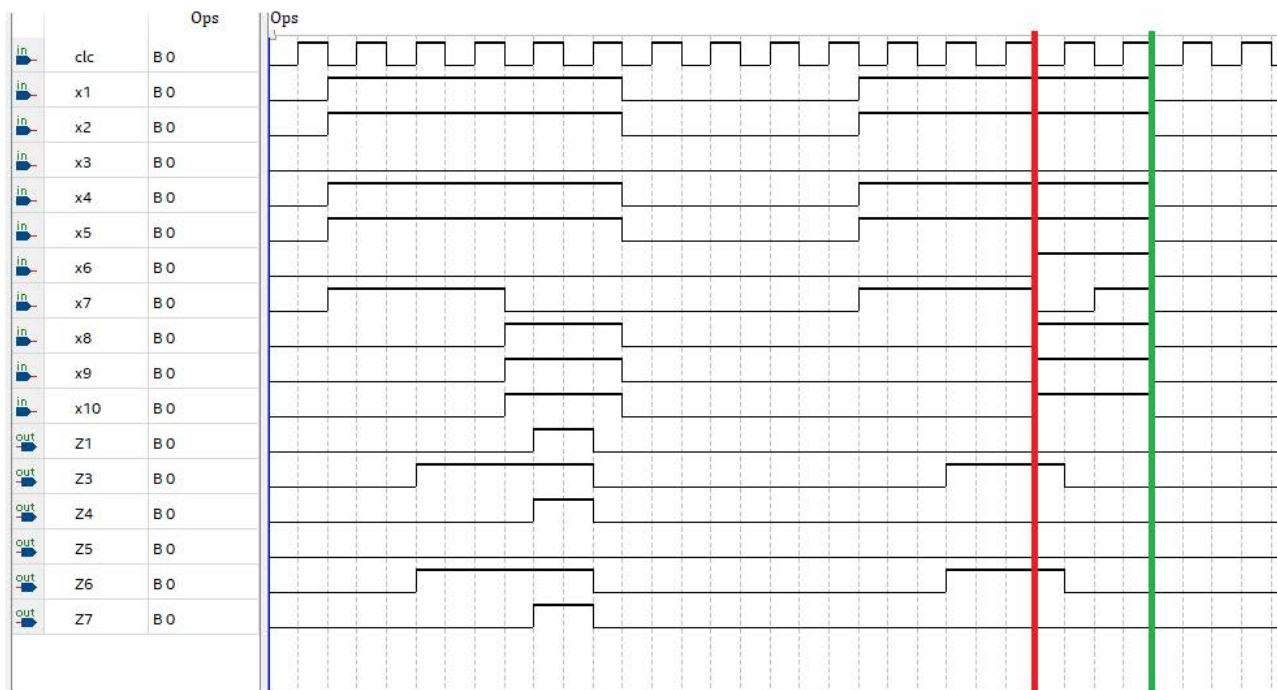


Figure 5. A finite-state machine operation simulation. The red line in the Figure indicates the moment of completion of the finite state machine operation when the route is set, followed by its closure and return to the initial state. The green line in the Figure characterizes the moment of completion of the transfer of the system to the protective state (from the red line) and back (from the green line) upon receipt of input data that do not correspond to the logic of the system operation.

Table 11. Parameters of the finite-state machine designed.

Family	Cyclone V
Device	5CEBA2F17A7
Logic utilization (in ALMs)	14
Total registers	16
Total pins	17

An internal control scheme is synthesized for the resulting FSM. The error correction code used is the modulo $M = 4$ with weights $[w_1, w_2, w_3] = [1, 2, 3]$. Since the weighting coefficients form a series of natural numbers and $M = m + 1$, where m is the number of digits in the data vector [24], this code will be able to detect all single errors in the data vector.

The weights of each data vector can be found by using the formula $\sum_{i=1}^m w_i f_i$, where f_i represents the value of the i -th function that describes the information character. Then, the smallest non-negative deduction modulo $M = 4$ should be found, and the resulting number should be represented in binary form. Table 12 lists the values of the control functions for the outputs of the memory cells.

Table 12. Finding control functions at memory cell outputs.

Decimal Number of a Binary Vector	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}y_1y_2y_3$	Y_{D1}	Y_{D2}	Y_{D3}	r	$r \pmod{4}$	g_1	g_2
0	0000000000 000	0	0	1	3	3	1	1
1	0000000000 001	0	0	1	3	3	1	1
2	0000000000 010	0	0	0	0	0	0	0
3	0000000000 011	0	0	0	0	0	0	0
4	0000000000 100	0	0	1	3	3	1	1
5	0000000000 101	0	0	1	3	3	1	1
64	0000001000 000	0	0	1	3	3	1	1
65	0000001000 001	0	0	1	3	3	1	1
66	0000001000 010	0	0	0	0	0	0	0
67	0000001000 011	0	0	0	0	0	0	0
68	0000001000 100	0	0	1	3	3	1	1
69	0000001000 101	0	0	1	3	3	1	1
128	0000010000 000	0	0	1	3	3	1	1
129	0000010000 001	0	0	1	3	3	1	1
130	0000010000 010	0	0	0	0	0	0	0
131	0000010000 011	0	0	0	0	0	0	0
132	0000010000 100	0	0	1	3	3	1	1
133	0000010000 101	0	0	1	3	3	1	1
192	0000011000 000	0	0	1	3	3	1	1
193	0000011000 001	0	0	1	3	3	1	1
194	0000011000 010	0	0	0	0	0	0	0
195	0000011000 011	0	0	0	0	0	0	0
196	0000011000 100	0	0	1	3	3	1	1
197	0000011000 101	0	0	1	3	3	1	1
256	0000100000 000	0	0	1	3	3	1	1
257	0000100000 001	0	0	1	3	3	1	1
258	0000100000 010	0	0	0	0	0	0	0
259	0000100000 011	0	0	0	0	0	0	0
260	0000100000 100	0	0	1	3	3	1	1
261	0000100000 101	0	0	1	3	3	1	1
320	0000101000 000	0	0	1	3	3	1	1
321	0000101000 001	0	0	1	3	3	1	1
322	0000101000 010	0	0	0	0	0	0	0
323	0000101000 011	0	0	0	0	0	0	0
324	0000101000 100	0	0	1	3	3	1	1
325	0000101000 101	0	0	1	3	3	1	1
384	0000110000 000	0	0	1	3	3	1	1
385	0000110000 001	0	0	1	3	3	1	1
386	0000110000 010	0	0	0	0	0	0	0
387	0000110000 011	0	0	0	0	0	0	0
388	0000110000 100	0	0	1	3	3	1	1
389	0000110000 101	0	0	1	3	3	1	1
448	0000111000 000	0	0	1	3	3	1	1
449	0000111000 001	0	0	1	3	3	1	1

Table 12. Cont.

Decimal Number of a Binary Vector	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}y_1y_2y_3$	Y_{D1}	Y_{D2}	Y_{D3}	r	$r \pmod{4}$	g_1	g_2
450	0000111000 010	0	0	0	0	0	0	0
451	0000111000 011	0	0	0	0	0	0	0
452	0000111000 100	0	0	1	3	3	1	1
453	0000111000 101	0	0	1	3	3	1	1
6968	1101100111 000	0	0	0	0	0	0	0
6969	1101100111 001	0	0	0	0	0	0	0
6970	1101100111 010	1	0	0	1	1	0	1
6971	1101100111 011	0	0	0	0	0	0	0
6972	1101100111 100	1	0	0	1	1	0	1
6973	1101100111 101	0	0	0	0	0	0	0
6976	1101101000 000	0	0	0	0	0	0	0
6977	1101101000 001	0	1	0	2	2	1	0
6978	1101101000 010	0	1	0	2	2	1	0
6979	1101101000 011	0	0	0	0	0	0	0
6980	1101101000 100	0	0	0	0	0	0	0
6981	1101101000 101	0	0	0	0	0	0	0
7488	1110101000 000	0	0	0	0	0	0	0
7489	1110101000 001	0	1	1	5	1	0	1
7490	1110101000 010	0	0	0	0	0	0	0
7491	1110101000 011	0	1	1	5	1	0	1
7492	1110101000 100	0	0	0	0	0	0	0
7493	1110101000 101	0	0	0	0	0	0	0
7544	1110101111 000	0	0	0	0	0	0	0
7545	1110101111 001	0	0	0	0	0	0	0
7546	1110101111 010	0	0	0	0	0	0	0
7547	1110101111 011	1	0	1	4	0	0	0
7548	1110101111 100	0	0	0	0	0	0	0
7549	1110101111 101	1	0	1	4	0	0	0
-	A^{**}	0	0	0	0	0	0	0

Write out the input sets, where the functions $g_i = 1$:

$$g_1 = \{0, 1, 4, 5, 64, 65, 68, 69, 128, 129, 132, 133, 192, 193, 196, 197, 256, 257, 260, 261, 320, 321, 324, 325, 384, 385, 388, 389, 448, 449, 452, 453, 6977, 6978\}$$

$$g_2 = \{0, 1, 4, 5, 64, 65, 68, 69, 128, 129, 132, 133, 192, 193, 196, 197, 256, 257, 260, 261, 320, 321, 324, 325, 384, 385, 388, 389, 448, 449, 452, 453, 6970, 6972, 7489, 7491\}$$

Finding the encoder functions (Table 13).

Table 13. Finding the first encoder functions.

$Y_{D1} Y_{D2} Y_{D3}$	g^*_1	g^*_2
0 0 0	0	0
0 0 1	1	1
0 1 0	1	0
0 1 1	0	1
1 0 0	0	1
1 0 1	0	0
1 1 0	~	~
1 1 1	~	~

Represent the g^*_i functions in disjunctive normal form (DNF):

$$g^*_1 = \overline{Y_{D1}} \overline{Y_{D2}} \overline{Y_{D3}} \cup \overline{Y_{D1}} \overline{Y_{D2}} Y_{D3}$$

Similarly, the internal control circuit for the output converter is synthesized. For this purpose, a weighted code modulo $M = 8$ with weighting coefficients $[w_1, w_2, w_3, w_4, w_5, w_6, w_7] = [1, 2, 3, 4, 5, 6, 7]$ is used [24]. Table 14 displays the control function values for the output vector of the finite-state machine, similar to Table 12.

[illegible]

Table 14. Cont.

Decimal Number of a Binary Vector	$x_1x_2x_3x_4x_5x_6x_7x_8x_9x_{10}y_1y_2y_3$	z_1	z_2	z_3	z_4	z_5	z_6	z_7	r	$r \pmod{8}$	g_1	g_2	g_3
6970	1101100111 010	1	0	1	1	0	1	1	21	5	1	0	1
6971	1101100111 011	0	0	0	0	0	0	0	0	0	0	0	0
6972	1101100111 100	1	0	1	1	0	1	1	21	5	1	0	1
6973	1101100111 101	0	0	0	0	0	0	0	0	0	0	0	0
6976	1101101000 000	0	0	0	0	0	0	0	0	0	0	0	0
6977	1101101000 001	0	0	1	0	0	1	0	9	1	0	0	1
6978	1101101000 010	0	0	1	0	0	1	0	9	1	0	0	1
6979	1101101000 011	0	0	0	0	0	0	0	0	0	0	0	0
6980	1101101000 100	0	0	0	0	0	0	0	0	0	0	0	0
6981	1101101000 101	0	0	0	0	0	0	0	0	0	0	0	0
7488	1110101000 000	0	0	0	0	0	0	0	0	0	0	0	0
7489	1110101000 001	0	0	1	0	1	0	0	8	0	0	0	0
7490	1110101000 010	0	0	0	0	0	0	0	0	0	0	0	0
7491	1110101000 011	0	0	1	0	1	0	0	8	0	0	0	0
7492	1110101000 100	0	0	0	0	0	0	0	0	0	0	0	0
7493	1110101000 101	0	0	0	0	0	0	0	0	0	0	0	0
7544	1110101111 000	0	0	0	0	0	0	0	0	0	0	0	0
7545	1110101111 001	0	0	0	0	0	0	0	0	0	0	0	0
7546	1110101111 010	0	0	0	0	0	0	0	0	0	0	0	0
7547	1110101111 011	1	0	1	1	1	0	1	20	4	1	0	0
7548	1110101111 100	0	0	0	0	0	0	0	0	0	0	0	0
7549	1110101111 101	1	0	1	1	1	0	1	20	4	1	0	0
-	A **	0	0	0	0	0	0	0	0	0	0	0	0

Write out the input sets, where the functions $g_i = 1$:

$$g_1 = \{6970, 6972, 7547, 7549\}$$

$$g_2 = \emptyset$$

$$g_3 = \{6970, 6972, 6977, 6978\}$$

Find the encoder functions (Table 15). All unused sets are denoted as A***.

Table 15. Finding the first encoder functions.

$z_1z_2z_3z_4z_5z_6z_7$	g^*_1	g^*_2	g^*_3
0 0 0 0 0 0 0	0	0	0
0 0 1 0 0 1 0	0	0	1
0 0 1 0 1 0 0	0	0	0
1 0 1 1 0 1 1	1	0	1
1 0 1 1 1 0 1	1	0	0
A ***	~	~	~

Represent the g_i^* functions in full disjunctive normal form (DNF):

$$g_1^* = Z_1 \overline{Z_2} Z_3 Z_4 \overline{Z_5} Z_6 Z_7 \cup Z_1 \overline{Z_2} Z_3 Z_4 Z_5 \overline{Z_6} Z_7$$

$$g_2^* = 0$$

$$g_3^* = \overline{Z_1} \overline{Z_2} Z_3 \overline{Z_4} \overline{Z_5} Z_6 \overline{Z_7} \cup Z_1 \overline{Z_2} Z_3 Z_4 \overline{Z_5} Z_6 Z_7$$

Figure 6 shows the block diagram of an FSM with embedded control circuits.

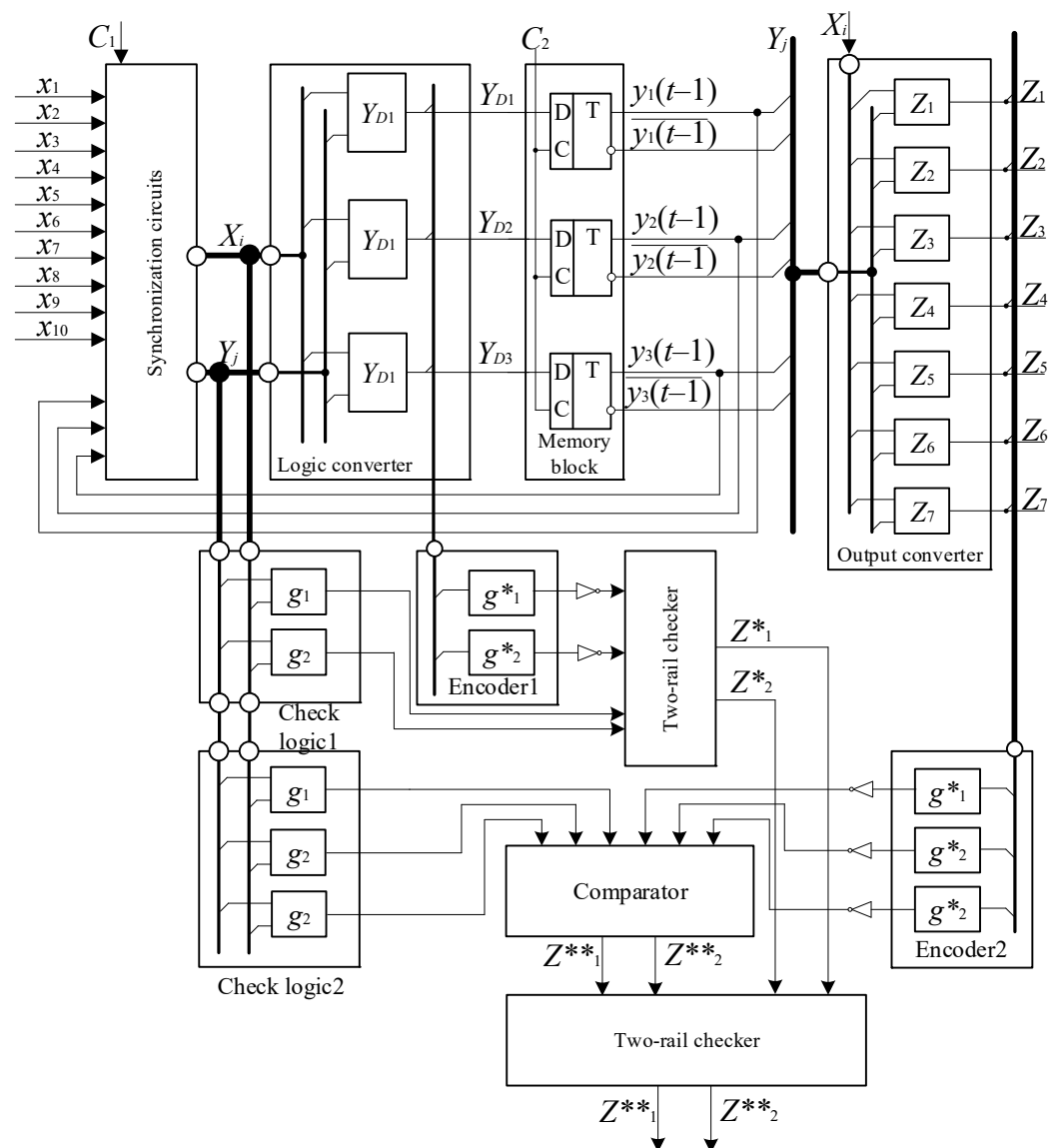


Figure 6. Block diagram of an FSM with embedded control circuits.

Note that the Z^*_1 and Z^*_2 functions are two-rail when operating correctly ($Z^*_1 = \overline{Z^*_2}$). In the event of a malfunction, both signals become equal. Tables 16 and 17 show the parameters of both synthesized embedded control circuits. They also take into account the synthesis of each control circuit on a separate chip. Synthesizing the control circuit on a single chip reduces the number of pins.

Table 16. ECC parameters for monitoring indoor units.

Family	Cyclone V
Device	5CEBA2F17A7
Logic utilization (in ALMs)	10
Total registers	0
Total pins	22

Table 17. ECC parameters for monitoring PV.

Family	Cyclone V
Device	5CEBA2F17A7
Logic utilization (in ALMs)	9
Total registers	0
Total pins	28

In order to create a system that can set routes for the entire station, it is essential to incorporate the unaccounted route states into the FSM design. It is worth noting that the design's complexity does not increase linearly. For instance, two routes necessitated six states and six distinct transition conditions. At the same time, only three flip-flops were necessary. Eight states are required for three routes. However, the number of flip-flops will remain unchanged. For the maximum number of routes in the example under consideration (26), 54 states are required. Six flip-flops are required to encode them. ECCs may become more complicated with the increase in the number of states.

7. Discussion

In the process of synthesizing train traffic control systems at intermediate stations, it is recommended that consideration be given to the numerous technical states of control objects. In this context, the control and monitoring objects encompass trackside railway automation and remote control equipment, including electric switch mechanisms, signals, train position control sensors, crossing automatics, and other pertinent components. The aforementioned facilities exhibit a limited range of technical states. Furthermore, the devices are interdependent, which results in their operation being closely integrated. All of these factors influence the various technical states of the system.

This paper outlines a methodology for synthesizing a train traffic control system. This is achieved by forming graphs for individual station traffic routes in a sequential manner and then combining them using the same states in different subgraphs. Some of the most evident benefits of this methodology are the ability to contemplate logical transitions between states, the potential to expand the range of states and incorporate protective and pre-failure states, the incorporation of functions for monitoring device operation, and, in the future, the control of analog parameters. This configuration renders the control system more akin to an adaptive automated control system rather than one that, in the event of a fault, exclusively enters protective states until the damage is rectified. This enables the potential for controlling the pre-failure states, thus preventing the attainment of protective failure states. This has the effect of enhancing the efficacy of the implementation of the train schedule and mitigating the risks of its disruption.

It is important to consider the limitations of the FSM model when used for the synthesis of control devices at railway stations. One such limitation is that the number of logical variables increases with the growth in the number of routes at stations, as well as with the growth in the number of states. Consequently, the challenge of implementing a set of interconnected FSMs arises. As a result, the implementation of an FSM for traffic control at large stations, comprising over 20–30 switches, is often challenging and, in some cases, unfeasible. Nevertheless, the number of small stations consistently outnumbers that of other station types within the railway networks. This is where the principles of typical system design for small intermediate stations can be developed.

With regard to the means of FSM protection and self-checking, it should be noted that this paper presents only the methodology for the synthesis of a protected FSM. This involves the encoding of states and the arrangement of embedded control circuits by the weight-based sum code. The selection of code is based on an evaluation of the capacity of the set of states and the lengths of data vectors formed by logical variables. This paper presents an illustrative example of the selection of the weight-based code for encoding. It should be noted that other stations and facilities may necessitate the utilization of a distinct weight-based code characterized by disparate parameters. However, it is always feasible

to construct a protected FSM. From the standpoint of developing a secure, commercially available solution, this research paper did not provide additional elaboration. Instead, it was assumed that the methods for implementing safe control systems, which have been well developed, would be used after synthesizing the control model [1,4,5,8,10].

In the future, an interesting solution would be the creation of an adaptive train traffic control system. Such a system would monitor analog parameters, control pre-failure states, and transmit data to the dispatching system, with automatic confirmation of changes in train traffic. However, these issues fall outside the scope of this paper.

8. Conclusions

This paper proposes a method for synthesizing finite-state machines for railway automation and remote control units. This method enables the synthesis of self-checking structures that can transit to pre-failure and protective states. These functions enable the integration of control and state monitoring tools, providing timely intervention in the event of device malfunctions. This reduces the likelihood of train schedule disruptions and failures.

The logical descriptions of the functioning and mutual alignment of intermediate station facility control systems represent the complexity of their synthesis. The issue of providing a logical description is less challenging for intermediate stations compared to those with a more complex topology. However, even in this case, it is possible to use typical implementations of finite-state machines for individual railway automation and remote control units.

The creation of an FSM to manage train traffic at stations may transpire in two distinct approaches. The first approach involves describing the conditions for transitioning between states for every possible object at the station and outlining the means of controlling them. It will be a single FSM. The second method involves creating models of FSMs for each automation unit. These models are then combined by constructing transition graphs, as demonstrated in this paper. This method is recommended since it enables the removal of states in individual device finite-state machines that are not necessary for analysis.

The potential for incorporating self-diagnostics and monitoring into these FSMs is worth considering. In order to monitor the operation of any railway automation installation and to document both pre-failure states and failures, internal states can be defined. This can be achieved, among other methods, by broadening a range of diagnostic parameters when fitting automation devices with external monitoring sensors. An instance of such integration is presented in [6]. Incorporating states into the transition graph of an FSM that considers pre-failure states and failures enables the subsequent implementation of both a control system and an embedded diagnostic system. This diagnostic system makes it possible to simulate the operation of the unit during its intended use. It facilitates the rapid identification and correction of any malfunction. Thus, the control system and diagnostic system are combined, offering the potential to create a digital copy of the control system [2]. Further research can be conducted in this area.

A drawback to consider is that when dealing with actual railway automation and remote control systems, the FSM graphs may contain a high number of states, especially if diagnostic functions are added. Therefore, it is essential to consider the issues of finite-state machine decomposition when synthesizing an automaton model for routes [25].

The synthesis of self-checking devices in railway automation and remote control with the possibility of transitioning to protective and pre-failure states enables the implementation of a train traffic control system resistant to external destabilizing factors.

Author Contributions: Conceptualization, A.V.P. and D.V.E.; methodology, D.V.E.; validation, D.V.E., D.N.K., D.K.B. and D.K.R.; formal analysis, D.G.P. and A.V.B.; investigation, A.V.P. and D.V.E.; writing—original draft preparation, A.V.P., E.M.M. and D.V.E.; writing—review and editing, D.V.E. and V.V.K.; visualization, A.V.P., R.B.A., E.M.M. and A.V.L.; supervision, D.V.E.; project administration, D.V.E. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Ministry of Science and Higher Education of the Russian Federation as part of the World-class Research Center program: Advanced Digital Technologies (contract No. 075-15-2022-311 dated 20 April 2022).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Theeg, G.; Vlasenko, S. *Railway Signalling & Interlocking*, 3rd ed.; PMC Media House GmbH: Leverkusen, Germany, 2020; p. 552.
2. Efanov, D.; Lykov, A.; Osadchy, G. Testing of Relay-Contact Circuits of Railway Signalling and Interlocking. In Proceedings of the 15th IEEE East-West Design & Test Symposium (EWDTS'2017), Novi Sad, Serbia, 29 September–2 October 2017; pp. 242–248. [\[CrossRef\]](#)
3. Bădău, F. Railway Interlockings—A Review of the Current State of Railway Safety Technology in Europe. *Promet Traffic Transp.* **2022**, *34*, 443–454. [\[CrossRef\]](#)
4. Dobiáš, R.; Kubátová, H. FPGA Based Design of Railway's Interlocking Equipment. In Proceedings of the EUROMICRO Symposium on Digital System Design, Rennes, France, 31 August–3 September 2004; pp. 467–473. [\[CrossRef\]](#)
5. Dobias, R.; Konarski, J.; Kubatova, H. Dependability Evaluation of Real Railway Interlocking Device. In Proceedings of the 2008 11th EUROMICRO Conference on Digital System Design Architectures, Methods and Tools, Parma, Italy, 3–5 September 2008; pp. 228–233. [\[CrossRef\]](#)
6. Efanov, D.V.; Pashukov, A.V.; Khóroshev, V.V. FPGA Self-Diagnosing and Testable Control Devices for Railway Switch Point. In Proceedings of the 2023 Seminar on Networks, Circuits and Systems (NCS), St. Petersburg, Russia, 29–30 November 2023; pp. 54–59. [\[CrossRef\]](#)
7. Heidmann, L. Smart Point Machines: Paving the Way for Predictive Maintenance. *Signal Draht* **2018**, *110*, 70–75.
8. Eriş, O.; Mutlu, İ. Design of Signal Control Structures Using Formal Methods for Railway Interlocking Systems. In Proceedings of the 11th International Conference on Control Automation Robotics & Vision, Singapore, 7–10 December 2010. [\[CrossRef\]](#)
9. Karolak, J.; Daszczuk, W.B.; Grabski, W.; Kochan, A. Temporal Verification of Relay-Based Railway Traffic Control Systems Using the Integrated Model of Distributed Systems. *Energies* **2022**, *15*, 9041. [\[CrossRef\]](#)
10. Sapozhnikov, V., VI. *Synthesis of Train Traffic Control Systems at Railway Stations with the Exception of Dangerous Failures*; Publishing house Nauka: Moscow, Russia, 2021; p. 229. (In Russian)
11. Mirabadi, A.; Yazdi, M.B. Automatic generation and verification of railway interlocking control tables using FSM and NuSMV. *Transp. Probl.* **2009**, *4*, 103–110.
12. Wang, J.; Zhang, X.; Shi, P.; Cao, B.; Wang, B. A DNA Finite-State Machine Based on the Programmable Allosteric Strategy of DNzyme. *Int. J. Mol. Sci.* **2023**, *24*, 3588. [\[CrossRef\]](#) [\[PubMed\]](#)
13. Potekhin, A.I. Logical Foundations of Group Traffic Control of Trains. *Autom. Remote Control.* **2020**, *81*, 883–896. [\[CrossRef\]](#)
14. Gavzov, D.V.; Sapozhnikov, V.V.; Sapozhnikov, V., VI. Methods for Providing Safety in Discrete Systems. *Autom. Remote Control.* **1994**, *55*, 1085–1122.
15. Bestemyanov, P.F. Methods of Providing Hardware Safety for Microprocessor Train Control Systems. *Russ. Electr. Eng.* **2020**, *91*, 531–536. [\[CrossRef\]](#)
16. Ma, W.-G.; Hei, X.-H. An Approach for Design and Formal Verification of Safety-Critical Software. In Proceedings of the International Conference on Computer Application and System Modeling (ICCASM 2010), Taiyuan, China, 22–24 October 2010. [\[CrossRef\]](#)
17. Dincel, E.; Eris, O.; Kurtulan, S. Automata-Based Railway Signaling and Interlocking System Design [Testing Ourselves]. *IEEE Antennas Propag. Mag.* **2013**, *55*, 308–319. [\[CrossRef\]](#)
18. Sapozhnikov, V., VI; Kononov, V.A. *Electrical Interlocking Systems*; Route: Moscow, Russia, 2002; p. 168. (In Russian)
19. Gordon, M.A.; Vasilenko, P.A.; Sedykh, D.V. Synthesis of Full Functional Check Programs for Train Traffic Management Systems on a Railway Station. In *Journal of Physics: Conference Series*. 13. Series. “Computer-Aided Technologies in Applied Mathematics”; IOP Publishing: Bristol, UK, 2020; p. 012013. [\[CrossRef\]](#)
20. Efanov, D.V.; Khoroshev, V.V.; Osadchy, G.V. Conceptual Foundations of the Synthesis of Safe Train Traffic Control Systems. *World Transp. Transp.* **2022**, *20*, 168–175. [\[CrossRef\]](#)
21. Efanov, D.V.; Khóroshev, V.V.; Osadchy, G.V. Principles of Safety Signalling and Traffic Control Systems Synthesis on Railways. In Proceedings of the 9th International Conference on Industrial Engineering, Applications and Manufacturing (ICIE), Sochi, Russia, 15–19 May 2022; pp. 634–638. [\[CrossRef\]](#)
22. Harris, D.M.; Harris, S.L. *Digital Design and Computer Architecture*; Morgan Kaufmann: San Francisco, CA, USA, 2012; p. 569.
23. Salauyou, V. Synthesis of High-Speed Finite State Machines in FPGAs by State Splitting. In Proceedings of the 15th IFIP International Conference on Computer Information Systems and Industrial Management (CISIM), Vilnius, Lithuania, 14–16 September 2016; pp. 741–751. [\[CrossRef\]](#)

24. Efanov, D.V.; Pashukov, A.V. Weight-Based Sum Codes with Arbitrary Modulus. In Proceedings of the IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIConRus), St. Petersburg, Russia, 25–28 January 2022; pp. 133–138. [[CrossRef](#)]
25. Agibalov, G.P.; Evtushenko, N.V. *Decomposition of Finite Automata*; Tomsk University Press: Tomsk, Russia, 1985; p. 127. (In Russian)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.